

The Complete Guide to Context in Golang: Efficient Concurrency Management



Jamal Kaksouri

Follow

15 min read · Jul 4, 2023



1.7K



15



Context



Introduction

Concurrency is a fundamental aspect of Go programming, and effectively managing concurrent operations is crucial for building robust and efficient applications. One of the key features that aids in achieving this is the Context package in Golang. Context provides a mechanism to control the lifecycle, cancellation, and propagation of requests across multiple goroutines. In this comprehensive guide, we will delve into the depths of context in Golang, exploring its purpose, usage, and best practices with real-world examples from the software industry.

The new features introduced in version 1.21 have been added to the article. These include:

func AfterFunc

func WithDeadlineCause

func WithTimeoutCause

func WithoutCancel

Table of Contents

1. What is Context?
2. Creating a Context
3. Propagating Context
4. Retrieving Values from Context
5. Cancelling Context
6. Timeouts and Deadlines

7. Context in HTTP Requests
8. Context in Database Operations
9. Best Practices for Using Context
10. Common Pitfalls to Avoid
11. Context and Goroutine Leaks
12. Using Context with Third-Party Libraries
13. Context(new features added in go1.21.0)
14. Conclusion
15. FAQs

1. What is Context?

Context is a built-in package in the Go standard library that provides a powerful toolset for managing concurrent operations. It enables the propagation of cancellation signals, deadlines, and values across goroutines, ensuring that related operations can gracefully terminate when necessary. With context, you can create a hierarchy of goroutines and pass important information down the chain.

Example: Managing Concurrent API Requests

Consider a scenario where you need to fetch data from multiple APIs concurrently. By using context, you can ensure that all the API requests are canceled if any of them exceeds a specified timeout.

```
package main
```

```
import (  
    "context"  
    "fmt"  
    "net/http"  
    "time"  
)  
  
func main() {  
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)  
    defer cancel()  
  
    urls := []string{  
        "https://api.example.com/users",  
        "https://api.example.com/products",  
        "https://api.example.com/orders",  
    }  
  
    results := make(chan string)  
  
    for _, url := range urls {  
        go fetchAPI(ctx, url, results)  
    }  
  
    for range urls {  
        fmt.Println(<-results)  
    }  
}  
  
func fetchAPI(ctx context.Context, url string, results chan<- string) {  
    req, err := http.NewRequestWithContext(ctx, "GET", url, nil)  
    if err != nil {  
        results <- fmt.Sprintf("Error creating request for %s: %s", url, err.Error())  
        return  
    }  
  
    client := http.DefaultClient  
    resp, err := client.Do(req)  
    if err != nil {  
        results <- fmt.Sprintf("Error making request to %s: %s", url, err.Error())  
        return  
    }  
    defer resp.Body.Close()  
  
    results <- fmt.Sprintf("Response from %s: %d", url, resp.StatusCode)  
}
```

Output:

```
Response from https://api.example.com/users: 200  
Response from https://api.example.com/products: 200  
Response from https://api.example.com/orders: 200
```

In this example, we create a context with a timeout of 5 seconds. We then launch multiple goroutines to fetch data from different APIs concurrently. The `http.NewRequestWithContext()` function is used to create an HTTP request with the provided context. If any of the API requests exceed the timeout duration, the context's cancellation signal is propagated, canceling all other ongoing requests.

2. Creating a Context

To create a context, you can use the `context.Background()` function, which returns an empty, non-cancelable context as the root of the context tree. You can also create a context with a specific timeout or deadline using `context.WithTimeout()` or `context.WithDeadline()` functions.

Example: Creating a Context with Timeout

In this example, we create a context with a timeout of 2 seconds and use it to simulate a time-consuming operation.

```
package main  
  
import (  
    "context"  
    "fmt"  
    "time"
```

```
)

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel()

    go performTask(ctx)

    select {
    case <-ctx.Done():
        fmt.Println("Task timed out")
    }
}

func performTask(ctx context.Context) {
    select {
    case <-time.After(5 * time.Second):
        fmt.Println("Task completed successfully")
    }
}
```

Output:

```
Task timed out
```

In this example, the `performTask` function simulates a long-running task that takes 5 seconds to complete. However, since the context has a timeout of only 2 seconds, the operation is terminated prematurely, resulting in a timeout.

3. Propagating Context

Once you have a context, you can propagate it to downstream functions or goroutines by passing it as an argument. This allows related operations to share the same context and be aware of its cancellation or other values.

Example: Propagating Context to Goroutines

In this example, we create a parent context and propagate it to multiple goroutines to perform concurrent tasks.

```
package main

import (
    "context"
    "fmt"
)

func main() {
    ctx := context.Background()

    ctx = context.WithValue(ctx, "UserID", 123)

    go performTask(ctx)

    // Continue with other operations
}

func performTask(ctx context.Context) {
    userID := ctx.Value("UserID")
    fmt.Println("User ID:", userID)
}
```

Output:

```
User ID: 123
```

In this example, we create a parent context using `context.Background()`. We then use `context.WithValue()` to attach a user ID to the context. The context

is then passed to the `performTask` goroutine, which retrieves the user ID using `ctx.Value()`.

4. Retrieving Values from Context

In addition to propagating context, you can also retrieve values stored within the context. This allows you to access important data or parameters within the scope of a specific goroutine or function.

Example: Retrieving User Information from Context

In this example, we create a context with user information and retrieve it in a downstream function.

```
package main

import (
    "context"
    "fmt"
)

func main() {
    ctx := context.WithValue(context.Background(), "UserID", 123)

    processRequest(ctx)
}

func processRequest(ctx context.Context) {
    userID := ctx.Value("UserID").(int)
    fmt.Println("Processing request for User ID:", userID)
}
```

Output:

Processing request for User ID: 123

In this example, we create a context using `context.WithValue()` and store the user ID. The context is then passed to the `processRequest` function, where we retrieve the user ID using type assertion and use it for further processing.

5. Cancelling Context

Cancellation is an essential aspect of context management. It allows you to gracefully terminate operations and propagate cancellation signals to related goroutines. By canceling a context, you can avoid resource leaks and ensure the timely termination of concurrent operations.

Example: Cancelling Context

In this example, we create a context and cancel it to stop ongoing operations.

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    ctx, cancel := context.WithCancel(context.Background())

    go performTask(ctx)

    time.Sleep(2 * time.Second)
    cancel()

    time.Sleep(1 * time.Second)
}
```

```
func performTask(ctx context.Context) {  
    for {  
        select {  
        case <-ctx.Done():  
            fmt.Println("Task cancelled")  
            return  
        default:  
            // Perform task operation  
            fmt.Println("Performing task...")  
            time.Sleep(500 * time.Millisecond)  
        }  
    }  
}
```

Output:

```
Performing task...  
Performing task...  
Task cancelled
```

In this example, we create a context using `context.WithCancel()` and defer the cancellation function. The `performTask` goroutine continuously performs a task until the context is canceled. After 2 seconds, we call the `cancel` function to initiate the cancellation process. As a result, the goroutine detects the cancellation signal and terminates the task.

6. Timeouts and Deadlines

Setting timeouts and deadlines is crucial when working with context in Golang. It ensures that operations complete within a specified timeframe and prevents potential bottlenecks or indefinite waits.

Example: Setting a Deadline for Context

In this example, we create a context with a deadline and perform a task that exceeds the deadline.

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    ctx, cancel := context.WithDeadline(context.Background(), time.Now().Add(2*time.Second))
    defer cancel()

    go performTask(ctx)

    time.Sleep(3 * time.Second)
}

func performTask(ctx context.Context) {
    select {
    case <-ctx.Done():
        fmt.Println("Task completed or deadline exceeded:", ctx.Err())
        return
    }
}
```

Output:

```
Task completed or deadline exceeded: context deadline exceeded
```

In this example, we create a context with a deadline of 2 seconds using `context.WithDeadline()`. The `performTask` goroutine waits for the context to be canceled or for the deadline to be exceeded. After 3 seconds, we let the program exit, triggering the deadline exceeded error.

7. Context in HTTP Requests

Context plays a vital role in managing HTTP requests in Go. It allows you to control request cancellation, timeouts, and pass important values to downstream handlers.

Example: Using Context in HTTP Requests

In this example, we make an HTTP request with a custom context and handle timeouts.

```
package main

import (
    "context"
    "fmt"
    "net/http"
    "time"
)

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel()

    req, err := http.NewRequestWithContext(ctx, "GET", "https://api.example.com/data", nil)
    if err != nil {
        fmt.Println("Error creating request:", err)
        return
    }

    client := http.DefaultClient
    resp, err := client.Do(req)
    if err != nil {
        fmt.Println("Error making request:", err)
    }
}
```

```
    return
}
defer resp.Body.Close()

// Process response
}
```

In this example, we create a context with a timeout of 2 seconds using `context.WithTimeout()`. We then create an HTTP request with the custom context using `http.NewRequestWithContext()`. The context ensures that if the request takes longer than the specified timeout, it will be canceled.

8. Context in Database Operations

Context is also useful when dealing with database operations in Golang. It allows you to manage query cancellations, timeouts, and pass relevant data within the database transactions.

Example: Using Context in Database Operations

In this example, we demonstrate how to use context with a PostgreSQL database operation.

```
package main

import (
    "context"
    "database/sql"
    "fmt"
    "time"

    _ "github.com/lib/pq"
)

func main() {
```

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()

db, err := sql.Open("postgres", "postgres://username:password@localhost/mydatabase")
if err != nil {
    fmt.Println("Error connecting to the database:", err)
    return
}
defer db.Close()

// Execute the database query with the custom context
rows, err := db.QueryContext(ctx, "SELECT * FROM users")
if err != nil {
    fmt.Println("Error executing query:", err)
    return
}
defer rows.Close()

// Process query results
}
```

In this example, we create a context with a timeout of 2 seconds using `context.WithTimeout()`. We then open a connection to a PostgreSQL database using the `sql.Open()` function. When executing the database query with `db.QueryContext()`, the context ensures that the operation will be canceled if it exceeds the specified timeout.

9. Best Practices for Using Context

When working with context in Golang, it is essential to follow some best practices to ensure efficient and reliable concurrency management.

Example: Implementing Best Practices for Context Usage

Here are some best practices to consider:

1. **Pass Context Explicitly:** Always pass the context as an explicit argument to functions or goroutines instead of using global variables. This makes it easier to manage the context's lifecycle and prevents potential data races.
2. **Use `context.TODO()`:** If you are unsure which context to use in a particular scenario, consider using `context.TODO()`. However, make sure to replace it with the appropriate context later.
3. **Avoid Using `context.Background()`:** Instead of using `context.Background()` directly, create a specific context using `context.WithCancel()` or `context.WithTimeout()` to manage its lifecycle and avoid resource leaks.
4. **Prefer Cancel Over Timeout:** Use `context.WithCancel()` for cancellation when possible, as it allows you to explicitly trigger cancellation when needed. `context.WithTimeout()` is more suitable when you need an automatic cancellation mechanism.
5. **Keep Context Size Small:** Avoid storing large or unnecessary data in the context. Only include the data required for the specific operation.
6. **Avoid Chaining Contexts:** Chaining contexts can lead to confusion and make it challenging to manage the context hierarchy. Instead, propagate a single context throughout the application.
7. **Be Mindful of Goroutine Leaks:** Always ensure that goroutines associated with a context are properly closed or terminated to avoid goroutine leaks.

Context in Real-World Scenarios

Context in Golang is widely used in various real-world scenarios. Let's explore some practical examples where context plays a crucial role.

Example: Context in Microservices

In a microservices architecture, each service often relies on various external dependencies and communicates with other services. Context can be used to propagate important information, such as authentication tokens, request metadata, or tracing identifiers, throughout the service interactions.

Example: Context in Web Servers

Web servers handle multiple concurrent requests, and context helps manage the lifecycle of each request. Context can be used to set timeouts, propagate cancellation signals, and pass request-specific values to the different layers of a web server application.

Example: Context in Test Suites

When writing test suites, context can be utilized to manage test timeouts, control test-specific configurations, and enable graceful termination of tests. Context allows tests to be canceled or skipped based on certain conditions, enhancing test control and flexibility.

10. Common Pitfalls to Avoid

1. Not propagating the context — Child functions need the context passed to them in order to honor cancelation. Don't create contexts and keep them confined to one function.
2. Forgetting to call cancel — When done with a cancelable context, call the cancel function. This releases resources and stops any associated goroutines.
3. Leaking goroutines — Goroutines started with a context must check the Done channel to exit properly. Otherwise they risk leaking when the context is canceled.

4. Using basic context. Background for everything — Background lacks cancelation and timeouts. Use the WithCancel, WithTimeout, or WithDeadline functions to add control.
5. Passing nil contexts — Passing nil instead of a real context causes panics. Make sure context is never nil when passing it.
6. Checking context too early — Don't check context conditions like Done() early in an operation. This risks canceling before the work starts.
7. Using blocking calls — Blocking calls like file/network IO should be wrapped to check context cancellation. This avoids hanging.
8. Overusing contexts — Contexts are best for request-scoped operations. For globally shared resources, traditional patterns may be better.
9. Assuming contexts have timeouts — The context.Background has no deadline. Add timeouts explicitly when needed.
10. Forgetting contexts expire — Don't start goroutines with a context and assume they will run forever. The context may expire.

11. Context and Goroutine Leaks

Contexts in Go are used to manage the lifecycle and cancellation signaling of goroutines and other operations. A root context is usually created, and child contexts can be derived from it. Child contexts inherit cancellation from their parent contexts.

Get Jamal Kaksouri's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

If a goroutine is started with a context, but does not properly exit when that context is canceled, it can result in a goroutine leak. The goroutine will persist even though the operation it was handling has been canceled.

Here is an example of a goroutine leak due to improper context handling:

```
func main() {
    ctx := context.Background()

    go func(ctx context.Context) {
        for {
            select {
            case <-ctx.Done():
                // properly handling cancellation
                return
            default:
                // do work
            }
        }
    }(ctx)

    time.Sleep(1 * time.Second)

    cancel() // cancel the context
}

func cancel() {
    ctx, cancel := context.WithCancel(context.Background())
    cancel() // cancel the context
}
```

In this example, the goroutine started with the context does not properly exit when that context is canceled. This will result in a goroutine leak, even though the main context is canceled.

To fix it, the goroutine needs to call the context's `Done()` channel when the main context is canceled:

```
func main() {
    ctx, cancel := context.WithCancel(context.Background())

    go func(ctx context.Context) {
        for {
            select {
            case <-ctx.Done():
                return // exit properly on cancellation
            default:
                // do work
            }
        }
    }(ctx)

    time.Sleep(1 * time.Second)

    cancel()
}
```

Now the goroutine will cleanly exit when the parent context is canceled, avoiding the leak. Proper context propagation and lifetime management is key to preventing goroutine leaks in Go programs.

12. Using Context with Third-Party Libraries

Sometimes we need using third-party packages. However, many third-party libraries and APIs do not natively support context. So when using such libraries, you need to take some additional steps to integrate context usage properly:

- Wrap the third-party APIs you call in functions that accept a context parameter.
- In the wrapper function, call the third-party API as normal.
- Before calling the API, check if the context is done and return immediately if so. This propagates cancellation.

- After calling the API, check if the context is done and return immediately if so. This provides early return on cancellation.
- Make sure to call the API in a goroutine if it involves long-running work that could block.
- Define reasonable defaults for context values like timeout and deadline, so the API call isn't open-ended.

[Open in app](#) ↗[Sign up](#)[Sign in](#)**Medium** Search Write

```
func APICall(ctx context.Context, args) {  
  
    // check for cancellation  
    if ctx.Done() {  
        return  
    }  
  
    // call API in goroutine  
    go func() {  
        result := thirdPartyAPI(args)  
  
        // check for cancellation  
        if ctx.Done() {  
            return  
        }  
  
        handleResults(result)  
    }()  
}
```

This provides context integration even with APIs that don't natively support it. The key points are wrapping API calls, propagating cancellation, using goroutines, and setting reasonable defaults.

13. Context(new features added in go1.21.0)

func AfterFunc

Managing cleanup and finalization tasks is an important consideration in Go, especially when dealing with concurrency. The context package provides a useful tool for this through its `AfterFunc` function.

`AfterFunc` allows you to schedule functions to run asynchronously after a context ends. This enables deferred cleanup routines that will execute reliably once some operation is complete.

For example, imagine we have an API server that needs to process incoming requests from a queue. We spawn goroutines to handle each request:

```
func handleRequests(ctx context.Context) {  
    for {  
        req := queue.Get()  
        go process(req)  
  
        if ctx.Done() {  
            break  
        }  
    }  
}
```

But we also want to make sure any pending requests are processed if `handleRequests` has to exit unexpectedly. This is where `AfterFunc` can help.

We can schedule a cleanup function to run after the context is cancelled:

```
ctx, cancel := context.WithCancel(context.Background())  
  
stop := context.AfterFunc(ctx, func() {
```

```
// Process remaining queue
})

go handleRequests(ctx)

// Later when done...
cancel()
stop() // Prevent cleanup
```

Now our cleanup logic will run after the context ends. But since we call `stop()`, it is canceled before executing.

`AfterFunc` allows deferred execution tied to a context's lifetime. This provides a robust way to build asynchronous applications with proper finalization.

func WithDeadlineCause

When using contexts with deadlines in Go, timeout errors are common — a context will routinely expire if an operation takes too long. But the generic “context deadline exceeded” error lacks detail on the source of the timeout.

This is where `WithDeadlineCause` comes in handy. It allows you to associate a custom error cause with a context's deadline:

```
ctx, cancel := context.WithDeadlineCause(ctx, time.Now().Add(100*time.Millisecond),
    errors.New("RPC timeout"))
defer cancel()

// Simulate work
time.Sleep(200 * time.Millisecond)

// Print the error cause
fmt.Println(ctx.Err()) // prints "context deadline exceeded: RPC timeout"
```

Now if the deadline is exceeded, the context's `Err()` method will return:

“context deadline exceeded: RPC timeout”

This extra cause string gives critical context on the source of the timeout. Maybe it was due to a backend RPC call failing, or a network request timing out.

Without the cause, debugging the timeout requires piecing together where it came from based on call stacks and logs. But `WithDeadlineCause` allows directly propagating the source of the timeout through the context.

Timeouts tend to cascade through systems — a low-level timeout bubbles up to eventually become an HTTP 500. Maintaining visibility into the original cause is crucial for diagnosing these issues.

`WithDeadlineCause` enables this by letting you customize the deadline exceeded error with contextual details. The error can then be inspected at any level of the stack to understand the timeout source.

func withTimeoutCause

Managing timeouts is an important aspect of writing reliable Go programs. When using context timeouts, the error “context deadline exceeded” is generic and lacks detail on the source of the timeout.

The `WithTimeoutCause` function addresses this by allowing you to associate a custom error cause with a context's timeout duration:

```
ctx, cancel := context.WithTimeoutCause(ctx, 100*time.Millisecond,
    errors.New("Backend RPC timed out"))
```

Now if that `ctx` hits the timeout deadline, the context's `Err()` will return:

“context deadline exceeded: Backend RPC timed out”

This provides critical visibility into the source of the timeout when it propagates up a call stack. Maybe it was caused by a slow database query, or a backend RPC service timing out.

Without a customized cause, debugging timeouts requires piecing together logs and traces to determine where it originated. But `WithTimeoutCause` allows directly encoding the source of the timeout into the context error.

Some key benefits of using `WithTimeoutCause`:

- Improved debugging of cascading timeout failures
- Greater visibility into timeout sources as errors propagate
- More context for handling and recovering from timeout errors

`WithTimeoutCause` gives more control over timeout errors to better handle them programmatically and debug them when issues arise.

func WithoutCancel

In Go, contexts form parent-child relationships — a canceled parent context will propagate down and cancel all children. This allows canceling an entire tree of operations.

However, sometimes you want to branch off a child context and detach it from the parent's lifetime. This is useful when you have a goroutine or operation that needs to keep running independently, even if the parent context is canceled.

For example, consider a server handling incoming requests:

```
func server(ctx context.Context) {  
    for {  
        req := waitForRequest(ctx)  
        go handleRequest(ctx, req) // child ctx  
  
        if ctx.Done() {  
            break  
        }  
    }  
}
```

If the parent ctx is canceled, the handleRequest goroutines will be abruptly canceled as well. This may interrupt requests that are mid-flight.

We can use `WithoutCancel` to ensure the handler goroutines finish:

```
func server(ctx context.Context) {  
  
    for {  
        req := waitForRequest(ctx)  
        handlerCtx := context.WithoutCancel(ctx)  
        go handleRequest(handlerCtx, req) // won't be canceled  
  
        if ctx.Done() {  
            break  
        }  
    }  
}
```

```
}  
}
```

Now the parent can be canceled, but each handler finishes independently.

`WithoutCancel` lets you selectively extract parts of a context tree to isolate from cancelation. This provides more fine-grained control over concurrency when using contexts.

Conclusion

In conclusion, understanding and effectively using context in Golang is crucial for developing robust and efficient applications. Context provides a powerful tool for managing concurrency, controlling request lifecycles, and handling cancellations and timeouts.

By creating, propagating, and canceling contexts, you can ensure proper handling of concurrent operations, control resource utilization, and enhance the overall reliability of your applications.

Remember to follow best practices when working with context and consider real-world scenarios where context can significantly improve the performance and reliability of your Golang applications.

With a strong grasp of context, you'll be well-equipped to tackle complex concurrency challenges and build scalable and responsive software systems.

Frequently Asked Questions (FAQs)

Q1: Can I pass custom values in the context?

Yes, you can pass custom values in the context using the `context.WithValue()` function. It allows you to associate key-value pairs with the context, enabling the sharing of specific data across different goroutines or functions.

Q2: How can I handle context cancellation errors?

When a context is canceled or times out, the `ctx.Err()` function returns an error explaining the reason for cancellation. You can handle this error and take appropriate actions based on your application's requirements.

Q3: Can I create a hierarchy of contexts?

Yes, you can create a hierarchy of contexts by using `context.WithValue()` and passing the parent context as an argument. However, it's important to consider the potential complexity and overhead introduced by nested contexts, and ensure proper management of the context lifecycle.

Q4: Is it possible to pass context through HTTP middleware?

Yes, you can pass context through HTTP middleware to propagate values, manage timeouts, or handle cancellation. Middleware intercepts incoming requests and can enrich the request context with additional information or modify the existing context.

Q5: How does context help with graceful shutdowns?

Context allows you to propagate cancellation signals to goroutines, enabling graceful shutdowns of concurrent operations. By canceling the context, you can signal goroutines to complete their tasks and release any resources they hold before terminating.

I hope this comprehensive guide on context in Golang has provided you with a deep understanding of its importance and practical usage. By leveraging the power of context, you can write efficient, scalable, and robust Golang applications.

Thank you for reading!

Please note that the examples provided in this article are for illustrative purposes only and may require additional code and error handling in real-world scenarios.

Happy Coding :).

[Golang](#)[Context](#)[Go Programming](#)[Go](#)[Golang Context](#)

Written by Jamal Kaksouri

415 followers · 20 following

Follow

A Software Engineer | Freelancer | Writer | Passionate about building scalable, performant apps and continuously learning

Responses (15)



Write a response

What are your thoughts?



Martin Kock

Sep 11, 2023



Well written article, I like it. But readers should be aware that some of the examples are incomplete and can't be used as-is. For example, in section 2, Creating a Context, you never wait for `performTask` to complete, either. You always just wait... [more](#)



13



1 reply

[Reply](#)



Muhammad Rivaldy

Aug 19, 2023



It's a good article Jamal. Now I'm interest to learn more deeply about context. I will go to the library docs after this.

One again, thanks!



9



1 reply

[Reply](#)



Sami Koskimäki

Aug 13, 2024 (edited)



Excellent article! Spotted a small typo though. You've used the following in couple of examples, which won't work. `ctx.Done()` returns a channel as you know.

...

```
if ctx.Done() {
```

```
break
```

```
}
```

... [more](#)

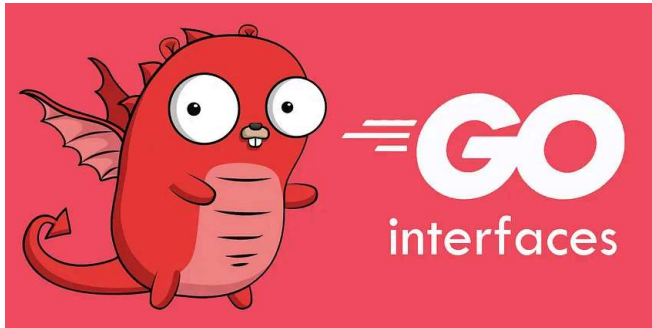


2

[Reply](#)

[See all responses](#)

More from Jamal Kaksouri



 Jamal Kaksouri

Understanding the Power of Go Interfaces: A Comprehensive Guide

The content of this article may be edited based on your feedback!

Feb 22, 2023  327  6



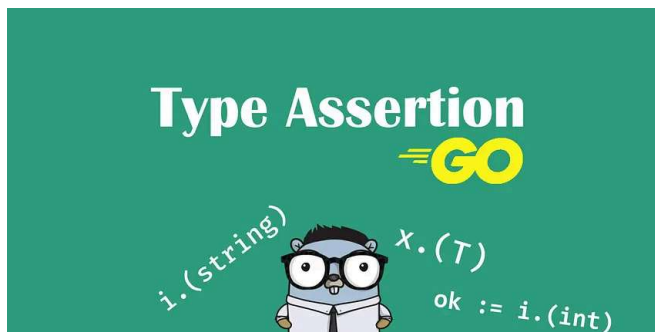
 Jamal Kaksouri

A Comprehensive Guide to Pointers in Go

Introduction

Feb 26, 2023  404  3





 Jamal Kaksouri

Mastering Type Assertion in Go: A Comprehensive Guide

In Go programming language, assertion is a mechanism to check the dynamic type of a...

Mar 3, 2023  302  3



 Jamal Kaksouri

Goroutines in Golang: Understanding and Implementing...

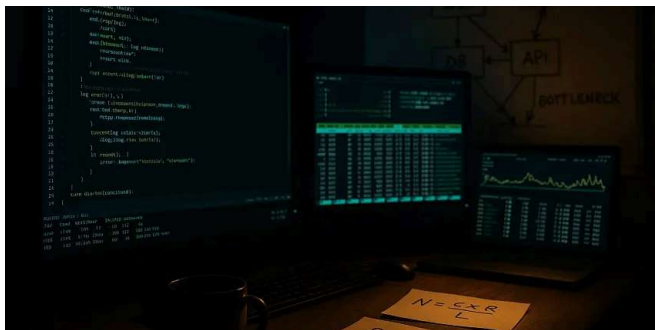
Golang, also known as Go, is a statically-typed and concurrent programming language that...

Feb 12, 2023  177



See all from Jamal Kaksouri

Recommended from Medium





DevLogics

13 GoLang Questions That Separate 10x Engineers from the...

Build systems that stay fast when traffic doubles. Fail quietly when something breaks...

★ Nov 3 🖱 17



In Stackademic by Maneesha Indrachapa

Advanced Go (Golang) Tricks Every Developer Should Master

Not a member? Read it for free here!

★ Sep 12 🖱 156 💬 2



⌚ Timer & 🕒 Ticker in Go



eapcell



Leapcell

Working with Scheduled Tasks in Go: Timer and Ticker

Explains Go's Timer and Ticker with usage, differences, and resource management tips.

Jun 30 🖱 9

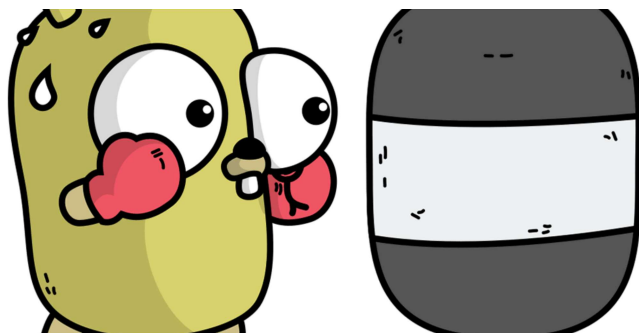


Ricardo Lüders

Mastering Go Internals: Runtime Internals and Unsafe Superpowers

Go gives you safety by default—but sometimes, safety isn't enough. In this secon...

Jun 10 🖱 3



Tech Guide

Why Senior Developers Are Choosing This Architecture for G...

If you've been around Go (Golang) projects long enough, you'll notice a pattern:



Puneet

5 Go Patterns That Will Save Your Kubernetes Clusters From...

Learn proven Go patterns that prevent resource disasters in Kubernetes. Real...

★ Jul 12 🖱️ 71 💬 5



★ Jul 5 🖱️ 10



See more recommendations